

In one line: VoiceOver and TalkBack read the **native** view tree, not your JSX: RN maps each prop onto UIAccessibility (iOS) or AccessibilityNodeInfo (Android). Prefer the web-aligned role + aria-* props (role wins over accessibilityRole); a few needs stay platform-specific — **live regions** are Android-only, iOS uses **announcements**; modal scoping is iOS-only.

What a prop becomes

RN prop	iOS (VoiceOver)	Android (TalkBack)
accessible	isAccessibilityElement	focusable
role · accessibilityRole	traits (button, header...)	node class / role description
aria-label	accessibilityLabel	contentDescription
aria-disabled/-checked/...	traits .notEnabled, .selected	node state (enabled, checked...)
aria-live="polite"	none — call announce	live region
aria-modal	accessibilityViewIsModal	none — hide siblings
aria-hidden	accessibilityElementsHidden	importantForA11y no-hide-descendants

Grouping a list row

before: 3 swipes, no context

after: accessible on the row + an action

1 swipe: "Alice, 2 new messages, button" · Actions: Archive

How it works

- Elements by default:** Text, TextInput, Switch; Pressable/touchables render an accessible View. A plain View is not. An accessible parent **groups** its children: one focus stop, label = the children's text joined.
- role takes web names (heading, img, slider, searchbox, list, presentation...); state/value via aria-checked ('mixed'), aria-expanded, aria-busy, aria-valuemin/max/now/text — same data as accessibilityState / accessibilityValue.
- Hint:** iOS reads it after a pause and users can switch hints off; Android **always** reads it. Keep it a short result ("Removes it"), never the label again.
- Actions:** accessibilityActions (activate, increment/decrement for adjustable, longpress/expand/collapse Android, magicTap/escape iOS, or custom) + onAccessibilityAction — the swipe-to-delete for screen readers.
- Focus order** = native tree order. Reorder: experimental_accessibilityOrder (list of nativeIDs). Move focus after navigation or an error: AccessibilityInfo.sendAccessibilityEvent(ref, 'focus') (setAccessibilityFocus deprecated in 0.85).
- Speak a change:** Android aria-live="polite" | "assertive" on the view that changes; iOS has no live regions → announceForAccessibility(msg) (iOS: ...WithOptions(msg, {queue: true}), announcementFinished event).
- User settings** (Promises + change events): both isScreenReaderEnabled, isReduceMotionEnabled; iOS isBoldTextEnabled, isReduceTransparencyEnabled, isInvertColorsEnabled; Android isHighTextContrastEnabled, getRecommendedTimeoutMillis (toasts, auto-dismiss).
- Text size:** allowFontScaling (default true), maxFontSizeMultiplier; read useWindowDimensions().fontScale. Fixed-height text rows clip at 200%.

VoiceOver vs TalkBack

	VoiceOver	TalkBack
hint	optional, after pause	always read
live change	announce API	aria-live region
modal scope	aria-modal	hide siblings (aria-hidden)
labelled by	—	aria-labelledby (nativeID)
extra gestures	escape (2-finger Z), magic tap	actions menu

Example

```
<Pressable role="button" aria-label="Delete draft"
  accessibilityHint="Removes it from this phone"
  aria-disabled={saving} disabled={saving}
  hitSlop={10} onPress={remove}> // 24pt icon, 44pt target
  <TrashIcon />
</Pressable>
<Pressable role="button" onPress={open} // one stop per row
  accessibilityActions={[{ name: 'archive', label: 'Archive' }]}
  onAccessibilityAction={e =>
    e.nativeEvent.actionName === 'archive' && archive()}>
  <Text>Alice</Text><Text>2 new messages</Text>
</Pressable>
<Text aria-live="polite">{status}</Text> // Android
useEffect(() => { // iOS (works on both)
  AccessibilityInfo.announceForAccessibility(status); }, [status]);
// test: only accessibility elements match, hidden ones excluded
expect(screen.getByRole('button', { name: 'Delete draft' }))
  .toBeDisabled();
```

WCAG basics

- Contrast** (1.4.3): text ≥ **4.5:1**, large text (18 pt / 14 pt bold) ≥ 3:1; UI parts and focus rings ≥ 3:1 (1.4.11). Never colour alone.
- Target size:** Apple **44×44 pt**, Material **48×48 dp**; WCAG 2.2 AA minimum 24×24 CSS px (2.5.8). hitSlop grows the touch area without changing layout.
- Respect **reduce motion** (swap slides for fades; Reanimated useReducedMotion()), don't auto-dismiss faster than the recommended timeout.

Testing

Manual: VoiceOver + Xcode **Accessibility Inspector** (audit); TalkBack + Android **Accessibility Scanner** (targets, contrast, labels). **Automated:** RNTL *ByRole / *ByLabelText / *ByHintText — a View matches by role only if accessible. testID is for E2E (iOS accessibilityIdentifier) — never spoken.

Traps

- !** An accessible card with a button inside: the button is no longer its own stop — expose it as an action.
- !** "Disabled" shown by opacity only: the screen reader still says it works — set disabled/aria-disabled.
- !** aria-live does nothing on iOS; aria-modal nothing on Android.
- !** Icon-only Pressable without aria-label: there is no name to read.

Remember

"Role, name, state — then focus, announce, scale, 44/48."